부록 · DB 관리 명령어 100선

Docker 4대 DB 통합관리 실전교재 Oracle + MariaDB + PostgreSQL + MongoDB

설정파일 · 영구저장 · 자동재시작 · 외부접속 · 사용자 생성 · 관리/접속 명령 비교

1. 목표와 전체 구조

Oracle, MariaDB, PostgreSQL, MongoDB를 하나의 Docker Compose 프로젝트에서 관리함. 컨테이너가 삭제되거나 서버가 재부팅되어도 DB 데이터가 남도록 호스트 data 폴더를 각각 연결하고, 각 DB의 표준 포트를 호스트에 공개하여 외부 PC와 Spring Boot에서 접속할 수 있게 구성함.

```text
~/docker-db/
├── docker-compose.yml
├── oracle/
│   └── data/
├── mariadb/
│   └── data/
├── postgresql/
│   └── data/
└── mongodb/
    └── data/
```

2. DB별 핵심 비교

DB	이미지 예	포트	컨테이너 데이터 위치	호스트 폴더
Oracle XE	gvenzl/oracle-xe:latest	1521	/opt/oracle/oradata	./oracle/data
MariaDB	mariadb:latest	3306	/var/lib/mysql	./mariadb/data
PostgreSQL 18+	postgres:latest	5432	/var/lib/postgresql (PGDATA는 버전별 하위 경로)	./postgresql/data
MongoDB	mongo:latest	27017	/data/db	./mongodb/data

3. 폴더 생성

```bash
mkdir -p ~/docker-db/oracle/data
mkdir -p ~/docker-db/mariadb/data
mkdir -p ~/docker-db/postgresql/data
mkdir -p ~/docker-db/mongodb/data
cd ~/docker-db
```

4. docker-compose.yml 완성본

```yaml
services:

  oracle:
    image: gvenzl/oracle-xe:latest
    container_name: oracle-xe
    restart: unless-stopped
    ports:
      - "1521:1521"
    environment:
      ORACLE_PASSWORD: "CHANGE_ME_ORACLE_ROOT"
      APP_USER: "devuser"
      APP_USER_PASSWORD: "CHANGE_ME_ORACLE_USER"
    volumes:
      - ./oracle/data:/opt/oracle/oradata
    networks:
      - db-network

  mariadb:
    image: mariadb:latest
    container_name: mariadb
    restart: unless-stopped
    ports:
      - "3306:3306"
    environment:
      MARIADB_ROOT_PASSWORD: "CHANGE_ME_MARIA_ROOT"
      MARIADB_DATABASE: "wizdata"
      MARIADB_USER: "devuser"
      MARIADB_PASSWORD: "CHANGE_ME_MARIA_USER"
    volumes:
      - ./mariadb/data:/var/lib/mysql
    networks:
      - db-network

  postgresql:
    image: postgres:latest
    container_name: postgresql
    restart: unless-stopped
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: "postgres"
      POSTGRES_PASSWORD: "CHANGE_ME_PG_ROOT"
      POSTGRES_DB: "wizdata"
    volumes:
      - ./postgresql/data:/var/lib/postgresql
    networks:
      - db-network

  mongodb:
    image: mongo:latest
    container_name: mongodb
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
```

```
      MONGO_INITDB_ROOT_USERNAME: "root"
      MONGO_INITDB_ROOT_PASSWORD: "CHANGE_ME_MONGO_ROOT"
    volumes:
      - ./mongodb/data:/data/db
    networks:
      - db-network

networks:
  db-network:
    driver: bridge
```

5. PostgreSQL 18+ 데이터 경로 주의

PostgreSQL 공식 Docker 이미지는 18부터 PGDATA가 버전별 경로(예: /var/lib/postgresql/18/docker)로 변경되었고, 공식 VOLUME은 /var/lib/postgresql로 변경됨. 따라서 최신 postgres:latest를 사용할 때는 호스트 폴더를 /var/lib/postgresql에 마운트하는 구성이 적합함. PostgreSQL 17 이하 이미지를 고정해서 사용할 경우에는 일반적으로 /var/lib/postgresql/data에 마운트함.

```yaml
# PostgreSQL 18+ (본 교재)
- ./postgresql/data:/var/lib/postgresql

# PostgreSQL 17 이하를 고정할 경우의 전형적 형태
- ./postgresql/data:/var/lib/postgresql/data
```

6. 설정파일 주요 항목 자세한 설명

설정	역할	설명
image	DB 이미지	어떤 DB 이미지/버전을 사용할지 지정함. 운영은 latest보다 검증된 버전 고정을 권장함.
container_name	컨테이너 이름	docker ps, logs, exec 등에서 사용할 사람이 읽기 쉬운 이름임.
restart: unless-stopped	자동 재시작	서버/Docker 재시작 후 컨테이너를 자동 기동하되 사용자가 명시적으로 중지한 경우는 유지함.
ports	포트 공개	호스트포트:컨테이너포트 형식. 외부 PC가 Docker DB에 접속할 수 있게 함.
environment	초기 환경변수	관리자 비밀번호, 초기 DB, 초기 사용자 등을 최초 초기화할 때 사용함.
volumes	영구 저장	컨테이너의 DB 데이터 경로를 호스트 data 폴더와 연결함.
networks	내부 네트워크	같은 Compose 프로젝트의 서비스가 서비스 이름으로 서로 통신할 수 있게 함.

7. 왜 data 폴더를 따로 만드는가

컨테이너는 삭제하고 다시 만들 수 있는 실행 단위임. 실제 DB 데이터는 호스트의 data 폴더에 남겨 두어야 컨테이너 재생성 시에도 데이터를 재사용할 수 있음.

```bash
컨테이너 삭제/재생성
        ↓
docker-compose.yml로 다시 생성
        ↓
기존 ./oracle/data
기존 ./mariadb/data
기존 ./postgresql/data
기존 ./mongodb/data
        ↓
DB 데이터 지속 사용
```

8. Docker 환경 영구 유지

```yaml
restart: unless-stopped

volumes:
  - ./DB별-data-폴더:/컨테이너-DB-데이터-경로
```

영구 운영의 핵심은 ① docker-compose.yml 보관, ② DB별 data 폴더 보관, ③ restart 정책 설정임. 단, data 폴더 자체를 rm -rf로 삭제하면 복구되지 않으므로 별도 백업이 필요함.

9. Compose 실행 순서

```bash
cd ~/docker-db

# YAML 문법 및 최종 설정 확인
docker compose config

# 이미지 다운로드
docker compose pull

# 백그라운드 실행
docker compose up -d

# 상태 확인
docker compose ps

# 전체 컨테이너 확인
docker ps
```

10. 외부접속 포트 열기 - Rocky Linux

```bash
sudo firewall-cmd --permanent --add-port=1521/tcp
sudo firewall-cmd --permanent --add-port=3306/tcp
sudo firewall-cmd --permanent --add-port=5432/tcp
sudo firewall-cmd --permanent --add-port=27017/tcp

sudo firewall-cmd --reload
sudo firewall-cmd --list-ports

sudo ss -lntp | grep -E '1521|3306|5432|27017'
```

보안상 공인 인터넷 전체에 DB 포트를 공개하지 말고, 가능한 경우 사내 LAN/VPN 또는 필요한 출발지 IP만 허용하는 것이 안전함.

11. DB별 외부접속 비교

DB	Host	Port	DB/Service	사용자 예
Oracle	Docker 서버 IP	1521	XEPDB1	devuser
MariaDB	Docker 서버 IP	3306	wizdata	devuser
PostgreSQL	Docker 서버 IP	5432	wizdata	postgres 또는 devuser
MongoDB	Docker 서버 IP	27017	admin / wizdata	root 또는 devuser

12. Oracle 접속 및 확인

```bash
# SYSDBA
docker exec -it oracle-xe sqlplus / as sysdba

# 애플리케이션 사용자
docker exec -it oracle-xe sqlplus devuser/CHANGE_ME_ORACLE_USER@XEPDB1

# SQL*Plus
SHOW PDBS;
ALTER SESSION SET CONTAINER=XEPDB1;
SELECT USER FROM DUAL;
```

13. MariaDB 접속 및 확인

```bash
docker exec -it mariadb mariadb -u root -p

SHOW DATABASES;
SELECT VERSION();
SELECT User,Host FROM mysql.user;

# devuser
docker exec -it mariadb mariadb -u devuser -p wizdata
```

14. PostgreSQL 접속 및 사용자 생성

```sql
docker exec -it postgresql psql -U postgres -d wizdata

SELECT version();
\l
\du
\dt

CREATE USER devuser WITH PASSWORD 'CHANGE_ME_PG_USER';
GRANT CONNECT ON DATABASE wizdata TO devuser;

\c wizdata
GRANT USAGE, CREATE ON SCHEMA public TO devuser;
```

15. MongoDB 접속 및 사용자 생성

```bash
docker exec -it mongodb mongosh   -u root   -p 'CHANGE_ME_MONGO_ROOT'   --authenticationDa
tabase admin

show dbs
use wizdata

db.createUser({
  user: "devuser",
  pwd: "CHANGE_ME_MONGO_USER",
  roles: [
    { role: "readWrite", db: "wizdata" }
  ]
})

db.getUsers()
```

16. 초기 환경변수 비교

항목	Oracle	MariaDB	PostgreSQL	MongoDB
관리자 암호	ORACLE_PASSWORD	MARIADB_ROOT_PASSWORD	POSTGRES_PASSWORD	MONGO_INITDB_ROOT_PASSWORD
초기 DB	기본 PDB/XEPDB1	MARIADB_DATABASE	POSTGRES_DB	사용 시 DB 생성
초기 사용자	APP_USER	MARIADB_USER	POSTGRES_USER는 초기 superuser 역할	Root env + 필요 시 별도 생성
사용자 암호	APP_USER_PASSWORD	MARIADB_PASSWORD	POSTGRES_PASSWORD	MONGO_INITDB_ROOT_PASSWORD
포트	1521	3306	5432	27017

17. 환경변수 변경 시 중요한 점

DB 이미지의 초기화용 환경변수는 일반적으로 빈 데이터 디렉터리에서 최초 초기화할 때 의미가 큼. 이미 data 폴더에 DB가 생성된 후 Compose 파일의 비밀번호나 초기 사용자 값을 바꿔도 기존 DB 내부 계정이 자동으로 같은 값으로 변경된다고 가정하면 안 됨. 기존 DB에서는 각 DB의 ALTER USER/계정 관리 명령을 사용함.

18. Windows 외부접속 테스트

```powershell
Test-NetConnection SERVER_IP -Port 1521
Test-NetConnection SERVER_IP -Port 3306
Test-NetConnection SERVER_IP -Port 5432
Test-NetConnection SERVER_IP -Port 27017
```

19. Spring Boot 연결 문자열 비교

DB	연결 문자열 예
Oracle	jdbc:oracle:thin:@SERVER_IP:1521/XEPDB1
MariaDB	jdbc:mariadb://SERVER_IP:3306/wizdata
PostgreSQL	jdbc:postgresql://SERVER_IP:5432/wizdata
MongoDB	mongodb://devuser:비밀번호@SERVER_IP:27017/wizdata

20. Docker 관리 + 컨테이너 + 접속 명령 30개

#	구분	용도	명령
1	공통	실행 컨테이너	docker ps
2	공통	모든 컨테이너	docker ps -a
3	공통	이미지 목록	docker images
4	Compose	설정 검증	docker compose config
5	Compose	이미지 다운로드	docker compose pull
6	Compose	전체 실행	docker compose up -d
7	Compose	서비스 상태	docker compose ps
8	Compose	전체 중지	docker compose stop
9	Compose	전체 시작	docker compose start
10	Compose	전체 재시작	docker compose restart
11	Compose	컨테이너/네트워크 제거	docker compose down
12	Oracle	시작	docker start oracle-xe
13	MariaDB	시작	docker start mariadb
14	PostgreSQL	시작	docker start postgresql
15	MongoDB	시작	docker start mongodb
16	Oracle	실시간 로그	docker logs -f oracle-xe
17	MariaDB	실시간 로그	docker logs -f mariadb
18	PostgreSQL	실시간 로그	docker logs -f postgresql
19	MongoDB	실시간 로그	docker logs -f mongodb
20	공통	자원 사용량	docker stats
21	공통	상세 정보	docker inspect <컨테이너명>
22	공통	포트 확인	docker port <컨테이너명>
23	공통	마운트 확인	docker inspect -f '{{json .Mounts}}' <컨테이너명>
24	공통	네트워크 목록	docker network ls
25	Oracle	SYSDBA 접속	docker exec -it oracle-xe sqlplus / as sysdba

#	구분	용도	명령
26	MariaDB	root 접속	docker exec -it mariadb mariadb -u root -p
27	PostgreSQL	psql 접속	docker exec -it postgresql psql -U postgres -d wizdata
28	MongoDB	mongosh 접속	docker exec -it mongodb mongosh -u root -p --authenticationDatabase admin
29	공통	컨테이너 쉘	docker exec -it <컨테이너명> bash
30	공통	디스크 사용량	docker system df

21. 컨테이너 삭제와 데이터 유지 이해

```bash
# 컨테이너/Compose 네트워크 제거
docker compose down

# 다시 생성
docker compose up -d
```

본 교재처럼 호스트 bind mount를 사용하면 docker compose down으로 컨테이너를 제거해도 호스트의 DB별 data 폴더는 그대로 남음. 단, 사용자가 직접 data 폴더를 삭제하면 데이터가 사라짐.

22. 주의해야 할 정리 명령

```bash
docker system prune -a
docker system prune -a --volumes
```

DB 서버에서는 prune 계열 명령을 습관적으로 실행하지 말고, 먼저 docker inspect로 실제 마운트와 백업 상태를 확인함. 특히 named volume을 함께 쓰는 다른 컨테이너가 있다면 --volumes 옵션은 더욱 신중해야 함.

23. 운영용 권장 방식

항목	개발/학습	운영 권장
이미지 태그	latest 가능	검증된 버전 고정
DB 포트	LAN 공개 가능	특정 IP/VPN 제한
비밀번호	Compose 직접 입력 가능	환경파일/Secret 사용
데이터	bind mount	정기 백업 + 스냅샷
재시작	unless-stopped	unless-stopped 또는 정책에 맞춤
업그레이드	pull 후 테스트	백업 → 호환성 확인 → 단계적 업그레이드

24. .env 파일로 비밀번호 분리하기

docker-compose.yml에 실제 비밀번호를 직접 적지 않고 .env 파일로 분리할 수 있음. .env 파일은 Git에 커밋하지 않는 것이 좋음.

```env
# .env
ORACLE_PASSWORD=강력한Oracle암호
ORACLE_APP_PASSWORD=강력한Oracle사용자암호
MARIADB_ROOT_PASSWORD=강력한MariaRoot암호
MARIADB_USER_PASSWORD=강력한Maria사용자암호
POSTGRES_PASSWORD=강력한Postgres암호
MONGO_ROOT_PASSWORD=강력한Mongo암호
```

```yaml
# docker-compose.yml 일부
environment:
  ORACLE_PASSWORD: "${ORACLE_PASSWORD}"

environment:
  MARIADB_ROOT_PASSWORD: "${MARIADB_ROOT_PASSWORD}"

environment:
  POSTGRES_PASSWORD: "${POSTGRES_PASSWORD}"

environment:
  MONGO_INITDB_ROOT_PASSWORD: "${MONGO_ROOT_PASSWORD}"
```

25. 최종 점검 체크

- docker compose config가 오류 없이 통과하는지 확인함.
- docker compose ps에서 4개 컨테이너가 Up인지 확인함.
- 1521, 3306, 5432, 27017 포트가 필요한 네트워크에서만 접근 가능한지 확인함.
- docker inspect로 각 DB의 data 마운트가 실제 호스트 폴더를 가리키는지 확인함.
- 컨테이너 재시작 후 기존 테스트 데이터가 그대로 남는지 확인함.
- 운영 전 각 DB를 별도로 백업하고 복원 테스트까지 수행함.

26. 참고 기준

본 교재는 2026년 9월 기준 Docker 공식 PostgreSQL 이미지의 PostgreSQL 18+ PGDATA/볼륨 경로 변경, MariaDB 공식 Docker 설치 방식, MongoDB의 27017 기본 포트 및 Docker 포트 매핑 방식을 반영하여 작성함.
